TELECOM Paris

IP PARIS

Direct access to mapped devices in memory

M2 SETI-B4 Embedded Linux

Maria Mushtaq
Maria.mushtaq@telecom-paris.fr
2022-2023

Institut Mines-Télécom

MMIO Devices

- Until now we have studied the methods of communication with peripherals connected to buses such as I2C, SPI, USB, etc. for which the kernel offers a communication API
- However, some peripherals (generally internal peripherals of an SoC or IPs loaded in an FPGA) are directly mapped in the physical address space
- The communication with these peripherals goes through classical memory reads and writes
- We speak of MMIO (Memory Mapped IO) peripherals

Reservation

```
struct resource * request_mem_region(resource_size_t start,
 resource_size_t n,
const char *name)
struct resource * devm_request_mem_region(struct device *dev,
resource_size_t start ,
resource_size_t n, const char *name)
```

- These two functions inform the kernel that our driver wishes to reserve the memory area starting at the physical address start and of length n bytes
- If this area is already reserved by another driver, they return NULL
- The list of reserved areas is available in the /proc/iomem file
- This step is not strictly necessary but recommended

Release

void release_mem_region(resource_size_t start, resource_size_t n);

- This function deletes the reservation

Memory Mapping

void int wait_event(wait_queue_head_t wait_queue , bool condition)
wait_event_interruptible(wait_queue_head_t wait_queue , bool condition)

- The device is mapped in physical memory. However, the kernel works in virtual address space
- It is therefore necessary to set up a range of virtual addresses which will be translated to the range of physical addresses corresponding to the device in order to access it

Memory Mapping

void __iomem *ioremap(resource_size_t offset, size_t size); **void** __iomem *devm_ioremap(**struct** device *dev,
resource_size_t offset , resource_size_t size);

- These functions set up a range of virtual addresses that are translated to the range of physical addresses offset to offset + size
- They return a pointer to the first virtual address or NULL if an error has occurred
- These functions ensure that the cache is disabled for access to these areas

Removal of memory mapping

void devm_iounmap(**struct** device *dev, **void** __iomem *addr); **void** iounmap(**volatile** **void** __iomem *addr)

- These two functions remove the memory mapping

Low Level Access

```
u8  __raw_readb(const volatile void __iomem *addr);
u16 __raw_readw(const volatile void __iomem *addr);
u32 __raw_readl(const volatile void __iomem *addr);

void __raw_writeb(u8  val, volatile void __iomem *addr);
void __raw_writew(u16 val, volatile void __iomem *addr);
void __raw_writel(u32 val, volatile void __iomem *addr);
```

- These primitives allow reads and writes
- No endianness conversion
- No protection against reordering at runtime (use the memory barriers described later)

High level access

```
u8  readb(const volatile void __iomem *addr);
u16 readw(const volatile void __iomem *addr);
u32 readl(const volatile void __iomem *addr);

void writeb(u8  val, volatile void __iomem *addr);
void writew(u16 val, volatile void __iomem *addr);
void writel(u32 val, volatile void __iomem *addr);
```

- These primitives allow reads and writes
- Read data are interpreted as little endian and possibly converted
- Protection against reordering at runtime (memory barriers are used by these functions)

Memory Barriers

- rmb(): Barrier in reading. All reads before (in the code) this barrier must be done before any read after this barrier
- wmb(): Write barrier. All writes before the barrier must be done before any write after the barrier
- mb(): Read and write barrier. All memory accesses (reads and writes) located before the barrier must be performed before any memory accesses located after the barrier
- Detailed explanations in
- Documentation/memory-barriers.txt

Plan

- Introduction
- Putting a task to sleep
- Wake up call
- Variant: exclusive waiting
- Conclusion

Exclusive waiting

- In some situations, several tasks may be waiting for an event but only one of these tasks will be able to process this event
- It is therefore unnecessary to wake them all up

Stand-by mode

- The functions seen before put the task in non-exclusive wait
- int

  wait_event_interruptible_exclusive(wait_queue_head wait_queue, bool

  condition) places the task in exclusive wait

Wake up call

- wake_up(wait_queue_head_t *q) and wake_up_interruptible(wait_queue_head_t *q) wake up all non-exclusive waiting tasks and a single exclusive waiting task (if any)
- wake_up_nr(wait_queue_head_t *q, int nr) and wake_up_interruptible_nr(wait_queue_head_t *q, int nr) wake up all non-exclusive pending tasks and at most nr exclusive pending tasks
- wake_up_all(wait_queue_head_t *q) and wake_up_interruptible_all(wait_queue_head_t *q) wake up all non-exclusive waiting tasks and all exclusive waiting tasks

Plan

- Introduction
- Putting a task to sleep
- Wake up call
- Variant: exclusive waiting
- Conclusion

Example od use in a driver

- A waiting list is declared in the foo_device structure to store processes waiting for data from the device
- In the foo_read callback function, the driver looks to see if data is currently available. If not, the process is put on hold
- In the interrupt handler called when data is available in the device, the driver wakes up the waiting processes